

Java Interview Bible — Extended Pack (50+ More Questions)

Add-on notes: final/this/super • inner classes • serialization • enums • generics • reflection • annotations • weak refs • GC tuning basics • CompletableFuture • plus a few “real-world” questions.

Updated: 2026-02-25

Use with: Core+Advanced v1

Goal: cover frequently asked “advanced core” topics

Table of Contents

- [A\) final / this / super](#)
- [B\) Inner / Nested classes](#)
- [C\) Enum tricks](#)
- [D\) Generics \(wildcards, PECS\)](#)
- [E\) Serialization](#)
- [F\) Reflection](#)
- [G\) Annotations](#)
- [H\) Weak/Soft/Phantom references](#)
- [I\) GC tuning basics](#)
- [J\) CompletableFuture](#)
- [K\) Extra frequently asked \(real-world\)](#)

Interview tip: When asked “Explain X”, speak in 3 steps: **(1) definition, (2) why/when used, (3) a small example.**

A) final / this / super

Q42. What does final mean for variable, method, and class?

1-minute answer

- **final variable:** can’t reassign reference/value
- **final method:** can’t be overridden
- **final class:** can’t be extended

Deep answer

For objects, final locks the reference, not the internal state (unless object is immutable). Final methods/classes are used for safety and design (e.g., immutable types).

Q43. What is this keyword?**1-minute answer**

- Refers to current object
- Used to resolve shadowing and call another constructor

Deep answer + example

```
class User {  
    private final String name;  
    User(String name) {  
        this.name = name; // field vs parameter  
    }  
    User() { this("guest"); } // constructor chaining  
}
```

Q44. What is super keyword and when must we use it?**1-minute answer**

- Access parent members
- Call parent constructor

Deep answer

If parent has no default constructor, child must call `super(args)`. Also used when parent and child both have same field/method name (to explicitly pick parent).

Q45. Can we have a final method in an abstract class?**Answer**

Yes. Abstract class can contain concrete methods too. Marking a method `final` means child classes cannot override it, which is useful for enforcing a stable algorithm step.

Where it's used

Template Method style: keep a final "template" method, and allow subclasses to implement specific steps via abstract/protected methods.

Q46. What is “shadowing” vs “overriding”?**Shadowing**

When a subclass declares a field with the same name as parent field, the subclass field **hides** the parent field. Field access depends on reference type.

Overriding

When subclass provides a new implementation of a parent method. Method calls are resolved at runtime based on object type.

B) Inner / Nested classes**Q47. What are types of nested classes in Java?****Types**

- **Static nested class**
- **Inner class** (non-static)
- **Local class** (inside a method)
- **Anonymous class**

Deep answer

Static nested has no implicit outer reference; inner class has an implicit reference to the outer instance which can cause memory leaks if misused.

Q48. Static nested class vs inner class (difference)

Static nested

- No outer instance needed
- Can have static members
- Good for helper/builder patterns

Inner (non-static)

- Needs outer instance
- Can access outer fields directly
- Holds implicit outer reference

Q49. Why can inner classes cause memory leaks?

Reason

An inner class instance keeps a hidden reference to the outer class instance. If inner object lives long (cached, static, background thread), it prevents GC of outer object.

Fix

Use a **static nested class** when you don't need outer reference, or carefully manage lifecycle/caches.

Q50. Anonymous class vs Lambda (common confusion)

Anonymous class

Creates a new class at runtime-like structure; can have state, can implement interfaces or extend classes.

Lambda

Used only for functional interfaces. It does not create a separate "named" class in the same way; it's mainly behavior.

C) Enum tricks

Q51. Why enum is better than constants (public static final int)?

1-minute answer

- Type-safe
- Fixed set of values
- Can have fields/methods

Deep answer

Enums prevent invalid values, allow behavior per constant, and work nicely with switch statements.

Q52. Can enum have constructor and fields?

Answer

Yes. Enum constructors are implicitly private. Each enum constant calls the constructor once.

Example

```
enum Role {  
    ADMIN(1), USER(2);  
    final int code;  
    Role(int code){ this.code = code; }  
}
```

Q53. Enum singleton: why is it considered safest?

Reason

Enum-based singleton is safe against **serialization** (JVM guarantees one instance) and harder to break with reflection (compared to normal classes).

Example

```
enum AppConfig {  
    INSTANCE;  
    String env = "prod";  
}
```

D) Generics (wildcards, PECS)

Q54. Why generics exist? What problem do they solve?

1-minute answer

- Compile-time type safety
- Remove casting
- Reusable code

Deep answer

Before generics, collections stored `Object` leading to runtime `ClassCastException`. Generics move errors to compile time.

Q55. What is type erasure?

Meaning

At runtime, generic type info (like `<String>`) is mostly removed. The JVM sees raw types (like `Object`) with casts inserted by compiler.

Impact

- Can't do `new T()`
- Can't have `List<String>.class`
- Some reflection/generic info limited

Q56. <?> vs <? extends T> vs <? super T>**Quick meanings**

- ? : unknown type
- ? extends T : some subtype of T (read-only-ish)
- ? super T : some supertype of T (write-friendly)

PECS rule

Producer Extends, Consumer Super: if you only read/produce T → use extends. if you write/consume T → use super.

Q57. Why List<Integer> is not a subtype of List<Number>?**Answer**

Generics are **invariant** in Java. If it were allowed, you could put a Double into a List<Integer> via List<Number> reference → type safety breaks.

Correct way

```
List<? extends Number> nums = List.of(1, 2, 3);
// nums.add(4); // not allowed (unknown exact subtype)
```

Q58. What are bounded type parameters?**Answer**

They restrict T to a certain hierarchy.

```
<T extends Number> double sum(List<T> list) { ... }
```

Why useful

It allows you to safely use methods of the bound (Number methods) on T.

E) Serialization

Q59. What is serialization and why is it used?**Definition**

Serialization converts an object into bytes so it can be stored (file/cache) or sent over network.

Use cases

- Caching objects
- Session replication
- Inter-process communication (legacy)

Q60. What is serialVersionUID?**Meaning**

It is a version identifier used during deserialization to ensure sender and receiver have compatible class versions.

Why define it

If you don't define it, JVM generates one from class structure. Even small code changes can break compatibility. Defining it gives control.

Q61. What does transient do?**Answer**

transient fields are not serialized. Useful for sensitive data (passwords) or derived fields that can be recomputed.

Important

On deserialization, transient fields become default values (null/0/false), unless you manually restore them.

Q62. How to make a singleton safe with serialization?**Problem**

Deserialization can create a new instance, breaking singleton.

Fix: readResolve()

```
private Object readResolve() {  
    return INSTANCE; // return existing singleton  
}
```

Q63. What are writeObject and readObject methods?**Answer**

They are hooks to customize serialization/deserialization, e.g., encrypting fields, handling transient/derived data, validation.

Note

Must keep signatures exactly as expected by serialization mechanism.

F) Reflection**Q64. What is reflection and when is it used?****Definition**

Reflection allows inspecting classes/methods/fields at runtime and invoking them dynamically.

Use cases

- Frameworks (Spring DI, ORM)
- Libraries (JSON serialization)
- Plugins / dynamic loading

Q65. Reflection disadvantages**Key drawbacks**

- Slower than direct calls
- Breaks encapsulation (setAccessible)
- Harder to maintain/debug

Interview angle

Use reflection mainly in frameworks; avoid in business code unless absolutely necessary.

Q66. How do you get Class object?**Ways**

```
Class<String> c1 = String.class;  
Class<?> c2 = obj.getClass();  
Class<?> c3 = Class.forName("java.lang.String");
```

Difference

`Class.forName` may trigger class loading/initialization. Useful when class name is known as string.

G) Annotations**Q67. What are annotations and why they are used?****Definition**

Metadata added to code. Tools/frameworks can read it to generate behavior, validation, configuration, etc.

Examples

- `@Override` (compile-time check)
- `@Deprecated` (warning)
- Spring: `@Component`, `@Autowired`

Q68. What is annotation retention policy?**Types**

- **SOURCE**: only in source code
- **CLASS**: in bytecode, not available at runtime
- **RUNTIME**: available via reflection at runtime

Why important

Framework annotations usually need **RUNTIME** so they can be read during execution.

Q69. What is @Target and @Retention?**@Target**

Where annotation can be applied (TYPE, METHOD, FIELD, PARAMETER...).

@Retention

How long it is kept (SOURCE/CLASS/RUNTIME).

H) Weak/Soft/Phantom references**Q70. Strong vs Weak vs Soft references (high-level)****Strong reference**

Normal reference. Object not collected until no strong refs exist.

Weak/Soft

Weak: GC can collect quickly when only weak refs remain. **Soft**: kept longer, collected under memory pressure (good for caches).

Q71. Why WeakHashMap exists and where used?**Answer**

WeakHashMap uses weak references for keys. When a key has no strong references elsewhere, entry can be removed by GC.

Use case

Metadata caches where you don't want cache entries to prevent GC of keys (like class metadata, listeners, etc.).

I) GC tuning basics**Q72. What is “stop-the-world” pause?****Meaning**

During certain GC phases, application threads pause so GC can safely move/mark objects.

Why it matters

In low-latency systems, long pauses are harmful. GC selection and heap sizing aim to reduce pause time.

Q73. How do you identify GC issues in production?**Signals**

- High latency spikes
- High CPU due to GC
- Frequent full GC
- OOM errors

Common approach

Enable GC logs, analyze pause time and frequency, check allocation rate, adjust heap sizes or reduce allocations.

Q74. Heap sizing basics: Xms vs Xmx**Answer**

-Xms is initial heap size; -Xmx is max heap. Setting them equal often reduces resizing overhead and improves stability.

Note

Heap too small → frequent GC; heap too big → longer full GC + memory waste. You want a balanced size.

J) CompletableFuture**Q75. What is CompletableFuture and why use it?****Definition**

It represents an async computation result that can be completed later and chained with callbacks.

Why useful

Cleaner async code vs manual threads. Supports chaining, combining multiple async tasks, error handling.

Q76. thenApply vs thenCompose**thenApply**

Transforms result: $T \rightarrow U$

```
cf.thenApply(x -> x + 1)
```

thenCompose

Flattens async: $T \rightarrow \text{CompletableFuture}<U>$

```
cf.thenCompose(x -> asyncCall(x))
```

Q77. How to combine two futures?**Answer**

Use thenCombine when both results needed.

```
CompletableFuture<Integer> c =  
    f1.thenCombine(f2, (a,b) -> a + b);
```

Alternative

allOf waits for all to finish (no typed result), anyOf returns first completed.

Q78. How to handle exceptions in CompletableFuture?**Common methods**

- exceptionally: fallback value
- handle: both success/error in one place

Example

```
cf.exceptionally(ex -> 0);  
cf.handle((val, ex) -> ex == null ? val : 0);
```

K) Extra frequently asked (real-world)

Q79. What is immutability and how to create an immutable class?

Rules

- Make class final
- Fields private final
- No setters
- Defensive copies for mutable fields

Why

Immutable objects are thread-safe by design, safe cache keys, and easier to reason about.

Q80. Why we override toString() in real projects?

Answer

For meaningful logging/debugging. Default toString prints class name + hash.

Tip

Do not print sensitive data (passwords, tokens) in toString.

Q81. What is marker interface? Examples?

Answer

A marker interface has no methods; it marks a class with a property used by JVM/frameworks.

Examples

Serializable, Cloneable

Q82. Cloneable: why cloning is tricky?**Problem**

`clone()` does shallow copy by default. Mutable fields may still be shared → bugs.

Preferred

Use copy constructors / factory methods for clarity and deep copy control.

Q83. What is Optional anti-pattern?**Anti-patterns**

- Using Optional for fields everywhere
- Calling `get()` without check
- Using Optional as method parameter heavily

Good usage

Primarily for return types to make “absence” explicit and avoid null chains.

Q84. equals contract: symmetry, transitivity, consistency**Key rules**

- Reflexive: $x \text{ equals } x$
- Symmetric: $x == y \Rightarrow y == x$
- Transitive: $x == y \text{ and } y == z \Rightarrow x == z$
- Consistent over time (if fields unchanged)

Why asked

Because incorrect `equals/hashCode` breaks `HashMap/HashSet` behavior in production.

Q85. Why “mutable key in HashMap” is dangerous?**Issue**

If key's state changes, its hashCode changes, so lookup goes to wrong bucket and entry becomes “lost”.

Fix

Use immutable keys (String, Integer, immutable custom classes).

Q86. What is ClassLoader and why important?**Answer**

Loads classes into JVM. Different classloaders can load same class name separately (important in app servers, plugins).

Common issue

ClassCastException due to same class loaded by different classloaders — they are treated as different types.

Q87. What is “happens-before” in Java memory model?**Meaning**

A rule that defines when one thread's writes become visible to another thread. Synchronization/volatile establish happens-before relationships.

Why asked

To evaluate understanding of visibility vs atomicity in concurrency.

Q88. What is ThreadLocal and when to use?**Answer**

ThreadLocal stores values per thread. Useful for request context, correlation id, user session context in legacy code.

Caution

In thread pools, threads are reused → must remove values (memory leak / wrong context). Use `remove()`.

Q89. What is CopyOnWriteArrayList and trade-off?**Answer**

It creates a new copy of array on write operations. Reads are very fast and thread-safe without locking.

Trade-off

Writes are expensive. Use when reads are frequent and writes are rare (listeners list).

Q90. Difference: synchronized vs ReentrantLock**synchronized**

Simpler, auto-release lock, less control.

ReentrantLock

More control: `tryLock`, fairness, interruptible lock, multiple conditions. Must unlock in finally.

Q91. What is `CompletableFuture` default thread pool and why should you care?**Answer**

Many async methods use **`ForkJoinPool.commonPool`** by default. If tasks block (I/O), it can starve the pool.

Best practice

Provide your own Executor for blocking calls: `supplyAsync(task, executor)`.

Q92. Difference between `Runnable` and `Callable`**`Runnable`**

No return value, cannot throw checked exceptions directly.

`Callable`

Returns value (Future), can throw checked exceptions.

Q93. What is `Future` vs `CompletableFuture`?**`Future`**

Represents result; mainly blocking `get()`. Limited chaining.

`CompletableFuture`

Non-blocking chaining, composition, combination, better error handling.

Q94. What is “backpressure” and does CompletableFuture provide it?**Meaning**

Backpressure is controlling producer speed when consumers are slow (avoid overload).

Answer

CompletableFuture alone doesn't provide backpressure. For streams with backpressure, reactive frameworks (Project Reactor/RxJava) are used.

Q95. Reflection vs Dependency Injection (why DI frameworks use reflection)**Answer**

DI frameworks create objects, set fields/constructors, and wire dependencies dynamically. Reflection enables reading annotations and invoking constructors/methods at runtime.

Trade-off

Great flexibility, but higher complexity and startup cost. Modern frameworks often optimize using caching and bytecode generation.

Q96. What is “annotation processing”?**Meaning**

Compile-time tool that reads annotations and generates code/resources (e.g., Lombok, MapStruct).

Why useful

Reduces boilerplate and avoids runtime reflection in some cases.

Q97. SoftReference vs WeakReference for caching**SoftReference**

Better for memory-sensitive caches: JVM tries to keep until memory pressure.

WeakReference

Collected more aggressively. Not ideal for “cache hit” scenarios; good for avoiding memory leaks with keys.

Q98. PhantomReference: what is it for?**Meaning**

Gives a way to know when an object is truly ready to be reclaimed, usually with ReferenceQueue.

Use case

Advanced resource cleanup beyond finalize (which is discouraged). Often used in off-heap resources handling.

Q99. Why finalize() is discouraged?**Problems**

- Unpredictable timing
- Can delay GC
- Can resurrect objects

Better alternatives

Try-with-resources, explicit close, Cleaner/PhantomReference patterns.

Q100. Real interview question: “What happens when you call a method?” (stack frame)**Short answer**

JVM pushes a new stack frame: local variables, operand stack, return address. On return, frame is popped.

Why it matters

Explains recursion stack overflow, why local variables are thread-safe, and method call performance.

Bonus: What I can add next if you want “full advanced pack”

- More concurrency: CompletableFuture patterns, ForkJoin, parallel streams pitfalls
- More JVM: class loading phases, GC collectors overview, allocation rate
- More Java language: records, sealed classes, pattern matching (if needed)
- Backend practical: HTTP status codes, REST idempotency, pagination, caching headers

End of extended pack.